AMERICAN UNIVERSITY OF BEIRUT
MAROUN SEMAAN FACULTY OF ENGINEERING & ARCHITECTURE

EECE 334
Project Spring 2026

Designing and Implementing a Domain-Specific Language for Multi-Agent AI Workflows

A. Introduction to Multi-Agent AI Workflows

In recent years, artificial intelligence systems have evolved from simple predictive models into autonomous software systems capable of reasoning, planning, and interacting with external tools. One of the most important developments in modern AI is the emergence of AI agents.

An AI agent is a software entity that can perceive information, reason about it, and perform actions to achieve a goal. Unlike traditional programs that follow rigid procedural logic, AI agents combine language understanding, reasoning, and tool usage to solve complex tasks. Modern agents are typically built on top of large language models (LLMs) that provide powerful natural language reasoning and generation capabilities.

LLMs such as those used in systems like GPT-4, Claude, or Llama have demonstrated the ability to interpret instructions, summarize information, write code, analyze text, and assist in decision making. However, these models alone are not sufficient to build complete intelligent systems. They must often interact with external systems such as databases, search engines, APIs, or computational tools. This is where the concept of tools becomes important.

A tool is an external capability that an AI agent can use to perform actions beyond language reasoning. For example, an agent might use a web search tool to retrieve information from the internet, a calculator tool to perform precise numerical computations, or an API tool to retrieve data from external services. When combined with a language model, these tools enable agents to perform tasks that require both reasoning and real-world interaction.

For example, consider an agent that must answer a question about current technological trends. The agent might perform the following sequence of actions:

1. Receive the user's request.
2. Use a web search tool to gather relevant information.
3. Analyze the retrieved information using an LLM.
4. Generate a summarized response.

This type of system is significantly more powerful than a static program because the agent can dynamically decide which tools to use and how to use them.

As AI systems grow more sophisticated, many applications require multiple agents collaborating together rather than a single agent performing all tasks. This leads to the concept of multi-agent workflows.

A multi-agent system is composed of several agents, each with a specialized role. Instead of attempting to solve an entire problem alone, agents divide responsibilities and communicate with each other. This design is similar to how teams of humans collaborate: different individuals perform different tasks based on their expertise.

For example, consider a system designed to produce analytical reports on emerging technologies. The workflow might involve several specialized agents:

• A research agent that gathers information from the web
• An analysis agent that interprets the collected data
• A writing agent that composes a final report

In such a system, the output produced by one agent becomes the input for another agent. The agents are executed in a specific order according to a workflow that describes how information flows through the system.

This orchestration of agents is known as an agent workflow.

In practice, real-world AI platforms now support such workflows through specialized frameworks such as LangChain and CrewAI. These frameworks allow developers to define agents, specify their tools, and coordinate their interactions. However, the complexity of these frameworks can make them difficult to understand at a fundamental level.

This project explores a different perspective: instead of using an existing framework, you will design your own domain-specific language (DSL) that describes multi-agent workflows. By designing a DSL, you create a high-level language that allows users to specify:

- what agents exist in a system
- what tools they can use
- what tasks they perform
- how agents interact with each other
- how data flows through the system

The purpose of the DSL is not to implement the full AI functionality itself, but to formally describe the structure of an AI workflow. In other words, the DSL acts as a declarative specification language for multi-agent systems.

Most importantly, the project demonstrates how complex domains—such as AI workflows—can be represented and analyzed using formal language design and compiler techniques, which are central topics in this course.

B. Project Overview

In this project you will design and implement a Domain-Specific Language (DSL) used to describe multi-agent AI workflows. The DSL will allow users to define:

- multiple agents
- tools used by agents
- tasks executed by agents
- typed inputs and outputs for tasks
- sequential execution of agents
- passing outputs between agents
- conditional execution
- loops
- variables with different types

Your system should be built using Python and will function as a DSL analyzer that:

1. Parses the DSL program
2. Performs semantic analysis
3. Reports syntax and semantic errors

The DSL models a system of AI agents collaborating sequentially. The language contains three main components.

- Agents: Agents define capabilities and tasks.
- Tasks: Tasks represent operations executed by an agent. Tasks may:
    - receive inputs
    - call tools
    - return outputs

- System Workflow: The system workflow coordinates the execution of agents and tasks. The workflow supports:
    - sequential execution
    - passing outputs between agents
    - conditional execution
    - loops
    - typed variables

- Supported Variable Types: The DSL supports the following types.

Type	Description
string	textual data
int	integer numbers
bool	boolean values
list	list of values

Examples:

```
string text
int counter
bool valid
list topics
```

C. Example DSL Program

The following example illustrates most features of the DSL.

```
agent Researcher {
    tool web_search
    tool llm

    task gather(string topic) -> string data {

        action: web_search(topic)
        action: llm("summarize results")
    }
}

agent Analyzer {
    tool llm
    task sentiment(string text) -> string result {
        action: llm("detect sentiment")
    }
}

system {
    list topics = ["AI","Robotics","Security"]
    int i = 0
    bool negative_found = false

    for t in topics {
        string data = run Researcher.gather(t)
        string sentiment = run Analyzer.sentiment(data)
        if sentiment == "negative" {
            negative_found = true
        }
        i = i + 1
    }
}
```

D. Data Passing Between Agents

Tasks may return values.

Example:

```
task gather(string topic) -> string data
```

The returned value can be assigned to a variable.

```
string data = run Researcher.gather(topic)
```

This variable can then be passed to another task or agent.

E. Conditional Execution

The system workflow supports conditional statements.

Example:

```
if sentiment == "negative" {

    negative_found = true

}
```

F. Loop Execution

The DSL supports loops for repeated execution.

Example:

```
for t in topics {

    string data = run Researcher.gather(t)

}
```

For simplicity, loops only iterate over a list.

G. Semantic Analysis

You must perform the following semantic validation.

Agent existence

Example error:
`run Editor.process(data)`
If `Editor` is not defined.

Task existence

Example:
`run Researcher.search(topic)`
Check that the task exists in the agent.

Tool declaration

Tools used in actions must be declared.

Variable declaration

Variables must be declared before use.
Example error:
`run Writer.write(data)`
If `data` was never declared.

Type checking

Verify type compatibility.
Example error:
`int topic = "AI"`
Type mismatch.

Parameter type checking

Example:
task gather(string topic)
The call must pass a string.

Symbol Tables

You must implement symbol tables such as:

```
Agents table
Tasks table per agent
Tools table per agent
System variables table
```

Example:

```
variables = {
   "topics": "list",
   "i": "int",
   "negative_found": "bool"
}
```

H. Project Deliverables

Phase 1 – Parsing – Due Sunday April 12 - Midnight

1. Grammar specification
2. FIRST and FOLLOW sets
3. Parsing table
4. Lexer implementation
5. LL1 Parser

Phase 2 – Semantic Analyzer - Due Sunday April 26 - Midnight

1. Semantic analyzer

For each phase, you should be providing in the addition to the above:

1. Example DSL programs demonstrating conditions, loops, and typed variables
2. Documentation describing design decisions and code

The use of AI tools is strictly prohibited. Students must be able to clearly explain and justify all aspects of their code and design. Any detected use of AI, in whole or in part, will result in an automatic grade of zero for the project, without exception

AMERICAN UNIVERSITY OF BEIRUT

MAROUN SEMAAN FACULTY OF ENGINEERING & ARCHITECTURE